Surfel Algorithm For Realtime Raytracing of Freeform Surfaces in Computer Graphics

Samuel Wiggins

Poolesville High School

2023

1 Introduction

Computer graphics are essential for applications such as real-time rendering, computer aided design, medical diagnostics, data visualizations, and mixed reality. For most of these fields three dimensional models have been used. Traditionally three dimensional models have been stored and represented as triangular models. However tesselating three dimensional models is limiting when simulating light, shadows, and more details. As an alternative to traditional rendering, ray tracing is used, which is a method of shooting rays from the camera to simulate light. While the traditional method of representing three dimensional models as multiple triangles is used for raytracing, raytracing is computationally expensive with intersection checks and memory increasing as the vertice count of a tessellated model increases. Freeform surfaces are proposed which is a representation of a three dimensional model as a mathematical function. Non uniform rational basis spline (NURBS) surfaces is a common function that is used for freeform surfaces, with frame rate higher using NURBS intersection operations than without. [2]

Recent developments in surfels allow faster ray tracing for three dimensional scenes. Surfels are point primitives without explicit connectivity, allowing the simplification of the representation of a three dimensional model. Since edges aren't needed, less memory is used to store data. Rays attempt to find the nearest surfel instead of intersecting with a surface, which improves performance. However there are occasionally image artifacts, as there is noise where there is the least amount of surfels generated. While layered depth images increase performance, they result at the cost of rendering accuracy. [1] Freeform surfaces allow faster surfel generation as you can use a mathematical equation, which avoids the extra computations needed for the searching and interpolation of vertices. Freeform surfaces ensure equidistant surfel generation as each NURBS

surface control point is a part of a grid formation. Surflets are best used for fast global illumination allowing real time frame rates when simulating light in a three dimensional scene. [4] Point primitives for freeform surfaces have been implemented as surflets, a theoretical method introduced as an algorithm to generate surfels on freeform surfaces. [3] Surflets is not focused on creating fast data retrieval or three dimensional object rendering with computer graphics. Instead, surflets are focused on creating an efficient representation of a higher dimensional function that can then be stored. Because of this advanced computer graphics algorithms aren't considered such as the rasterization or raytracing of surflets or path tracing approximation using resampled importance sampling. As a result this method is purely theoretical, as these algorithms are not applied to a practical real time renderer.

Current research on freeform surface intersections for ray tracing, uses two planes $P_1 : \vec{n_1} * \bar{x} - d_1 = 0$ and $P_2 : \vec{n_2} * \bar{x} - d_2 = 0$. Then an objective function system is created with $S(u, v)$ being the function of a non uniform rational basis spline surface defined as:

$$F(u,v) = \begin{pmatrix} n_1 \cdot S(u,v) - d_1 \\ n_2 \cdot S(u,v) - d_2 \end{pmatrix}$$

With this objective function newtonian iteration is performed with each step being $(u,v)_{k+1} = (u,v)_k - J^{-1}F((u,v)_k)$, with the jacobian J given by

$$J = \begin{pmatrix} n_1 \cdot \frac{\partial S(u,v)}{\partial u} & n_1 \cdot \frac{\partial S(u,v)}{\partial v} \\ n_2 \cdot \frac{\partial S(u,v)}{\partial u} & n_2 \cdot \frac{\partial S(u,v)}{\partial v} \end{pmatrix}$$

However computing the derivative function **∂S(u,v)** is costly as the mathematical definition of

derivation of a NURBS surface uses a large amount of operations that would be expensive to compute each frame for real time rendering. [5] Using a plane to represent the rays is also an inefficient way of representing the ray as the ray can be represented as a single vector, and also costly as the equation needs to be solved by finding n_1, n_2, d_1, and d_2.

This leads me to my research question, to what extent does the use of surflets for rendering NURBS surfaces impact the performance of freeform surface ray tracing. My paper proposes a method to address this research question which is a modification of traditional Newtonian iteration for ray tracing. This method incorporates surflets to improve guesses for the starting point of iteration. This method also uses a vector based representation of the ray instead of a plane based representation, which reduces memory usage on the device.

2 Methods

A novel renderer will be written in OpenGL and GLSL that uses raytracing. The renderer will be tested on one NURBS surface. Rays will be shot from the camera in the section of the camera view that has the least amount of rays. The ray will use the closest surflet that intersects with that ray vector which will then be used as a starting guess for newton iteration. These rays will use the intersection formula for non uniform rational basis spline surfaces to determine if a ray has successfully hit a three dimensional model in the scene. If a ray has intersected with a three dimensional model, then a surface point primitive is created from the intersection point, and that area of the screen is rendered with a color corresponding to the final UV position of the iteration.

2.1 Bounding Box Check For Each Ray

Function to get minimum and maximum ray vectors for bezier patch

---

B_0 : Minimum ray vector that can hit bezier patch
B_1 : Maximum ray vector that can hit bezier patch
p : 2d array of surflets of bezier surface

---

for i in n:
    for j in n:
        $p_{i,j_x} < B_{0_x} \rightarrow B_{0_x} = p_{i,j_x}$
        $p_{i,j_y} < B_{0_y} \rightarrow B_{0_y} = p_{i,j_y}$
        $p_{i,j_z} < B_{0_z} \rightarrow B_{0_z} = p_{i,j_z}$
        $p_{i,j_x} > B_{1_x} \rightarrow B_{1_x} = p_{i,j_x}$
        $p_{i,j_y} > B_{1_y} \rightarrow B_{1_y} = p_{i,j_y}$
        $p_{i,j_z} > B_{1_z} \rightarrow B_{1_z} = p_{i,j_z}$

This function is designed to get the bounding box for the possible ray positions. Essentially getting the minimum vector and maximum vector that a ray can be to have more than a 0% chance of colliding with the freeform surface. This allows us to immediately detect whether a ray hits a surface, allowing ray's that don't hit the surface to be immediately discarded to save processing power.

2.2 Getting Closest Surflet For Each Ray

Function to get closest point index to ray

---

d : Closest distance
I : Closest point index i for U
J : Closest point index j for V
O : origin

---

for i in n:
    for j in n:
        $v_{i,j} = p_{i,j} - O$
        $L = |\hat{v_i, j} - \hat{v}|$
        $if L < d$:
            $I = i$
            $J = j$
            $d = L$

This function is designed to get the closest surflet, a freeform surface point primitive, for each ray. This is so we can get color data from the nearest surflet, and create a starting guess for this paper's method of Newtonian iteration. This is essential as less iteration steps will be done in the newtonian iteration if the algorithm guesses the closest surfel correctly.

2.3 Newton Iteration

Iteration

U : start u position
V : start v position
O : origin
$\vec{v}_{target}$: target direction
δ : offset
ϵ : margin of error
n : tries

---

while(true):
$\vec{L} = S(U - \delta, V) - O$
$\Delta\vec{L} = |\hat{L} - \vec{v}_{target}|$
$\vec{R} = S(U + \delta, V) - O$
$\Delta\vec{R} = |\hat{R} - \vec{v}_{target}|$
$\vec{U} = S(U, V + \delta) - O$
$\Delta\vec{U} = |\hat{U} - \vec{v}_{target}|$
$\vec{D} = S(U, V - \delta) - O$
$\Delta\vec{D} = |\hat{D} - \vec{v}_{target}|$
$diff = \langle R - L, U - D \rangle$
$\Delta\langle U, V \rangle = \widehat{diff} * \delta$
$\langle U, V \rangle = \langle U, V \rangle + \Delta\langle U, V \rangle$
$U > min \vee U < max \rightarrow U = U - \Delta U$
$U > min \vee U < max \rightarrow U = V - \Delta V$
$\vec{v}_{current} = S(s_0, s_1) - O$
$\Delta\vec{v} = |\vec{v}_{current} - \vec{v}_{target}|$
$\vec{v} < \epsilon \rightarrow end$

For this algorithm the non uniform rational basis spline surface function is used which is defined as so:

$$\mathbf{S}(u, v) = \frac{\sum_{i=0}^{m} \sum_{j=0}^{n} N_{i,p}(u) N_{j,q}(v) w_{i,j} \mathbf{P}_{i,j}}{\sum_{i=0}^{m} \sum_{j=0}^{n} N_{i,p}(u) N_{j,q}(v) w_{i,j}},$$

where $N_{i,p}$ and $N_{j,q}$ are the B-spline basis functions defined as so:

$$N_{i,0}(t) = \begin{cases} 1 & \text{if } t_i \le t < t_{i+1} \text{ and } t_i < t_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

$$N_{i,j}(t) = \frac{t - t_i}{t_{i+j} - t_i} N_{i,j-1}(t) + \frac{t_{i+j+1} - t}{t_{i+j+1} - t_{i+1}} N_{i+1,j-1}(t),$$

$P_{i,j}$ are control points, and the $w_{i,j}$ is the weight of $P_{i,j}$.

Whenever possible the basis functions are precalculated and stored as memory to prevent calculations during runtime that could negatively impact framerate.

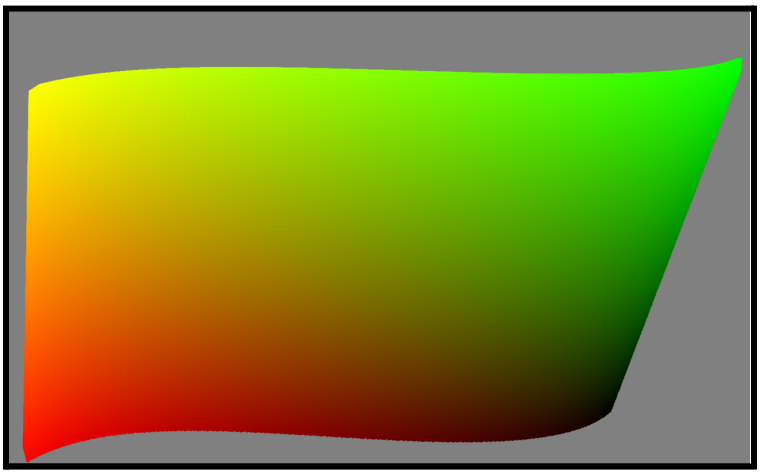
This function is designed to take a guess of where the ray hits from the recomputed surfels, and iterate until it reaches the correct point. Four vectors are generated including $\vec{R}$ the right direction for the point δ on the positive U axis, $\vec{L}$ the left direction for the point δ on the negative U axis, $\vec{U}$ the up direction for the point δ on the positive V axis, and $\vec{D}$ the down direction for the point δ on the negative V axis. These four vectors are then used to determine the direction of the vector. This direction is then normalized and multiplied by δ the offset which is added to the initial U and V. We then check to make sure the U and V variables are not out of bounds. We then check if the difference of the current ray direction and the target ray direction is less than the margin of error. If so the algorithm converges and the algorithm is successful in completing the intersection with the bezier freeform surface. To prevent stack overflows we have a maximum number of tries that the algorithm can iterate. If the amount of tries reaches the maximum we can conclude the algorithm diverges and there is no ray intersection.

2.4 Data Collection

Data will be collected from the participant's personal device such as average frame rate, CPU and GPU memory, and computer specifications. To collect average frame rate `glutGet(GLUT_ELAPSED_TIME)` from the OpenGL library is used which will allow retrieval of the current time at each frame. The time of each frame's previous time of each frame will be subtracted from the time of each frame to obtain frame rate. Using this information the average frame rate will be calculated which will be used to measure performance. The computer specifications consist of the machine's system, release, version, architecture, RAM, GPU, and CPU. Python libraries such as platform, psutil, cuda, and cpuinfo are used to extract these values from the machine. To collect current CPU memory for each frame, psutil is used.

All variables are uploaded to a google sheet using Google Drive API. Once data collection is finished the google sheet is downloaded as a .csv file for data analysis. Python libraries are used for data analysis with Numpy used for loading data from the .csv file, SkLearn used for data analysis, and Pandas used for data visualization. This will allow the establishment of association between independent variables such as output resolution, number of vertices, and computer specifications and output variables such as average frame rate and memory used on the CPU.

3 Results



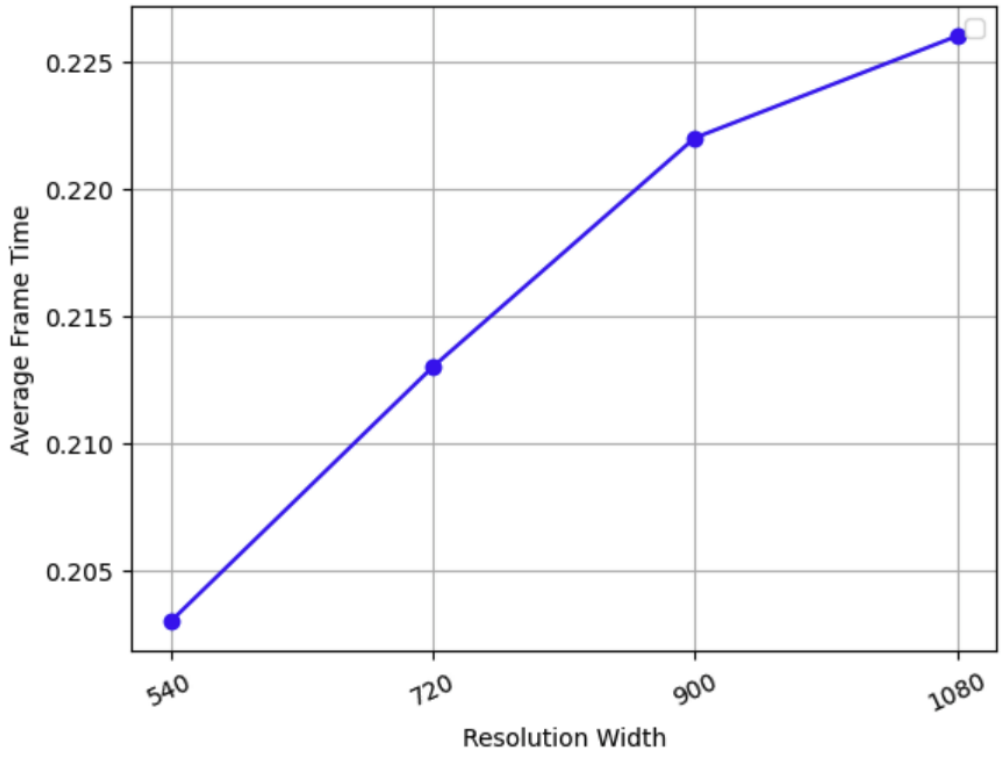
This image is the result of the algorithm ray tracing a basic freeform surface composed of 16 vertices. The surflets have color data based on the u,v coordinates which explains the gradient that the image produces.

3.1 Table

Resolution (Width x Height)	Average Frame Time	CPU Memory (in bytes)
960x540	0.23	190005248
1280x720	0.213	197365760
1600x900	0.222	207863808
1920x1080	0.226	210661376

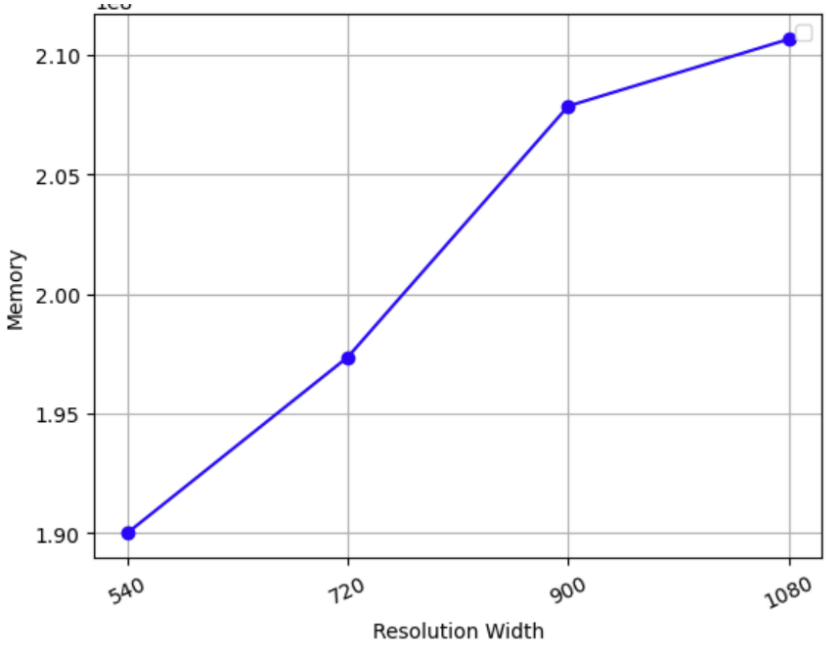
This is a table including the resolution as an independent variable and the average frame time and CPU memory as dependent variables. All data is gathered from a Windows 10 32 GB 12th Gen Intel(R) Core(TM) i7-12700H NVIDIA GeForce RTX 3060 computer.

3.2 Resolution and Average Frame Time



This graph shows that as the resolution increases the average frame time increases. This means that higher resolution renders are likely to be less performant than lower resolution renders. However the change in average frame rate gradually decreases as resolution increases. This implies that the difference between lower resolutions is noticeable, but the difference between higher resolutions negligibly impacts performance.

3.3 Resolution and CPU Memory



This graph shows that as the resolution increases the CPU memory increases. However a difference in ~20MB between 540p and 1080p resolution is not a significant difference for the consideration of resolution and CPU memory to correlate. Therefore we can conclude there is a negligible amount of correlation for resolution and CPU memory, so proof is provided that the memory of the algorithm scales with increasing resolution.

4 Conclusion

This algorithm performs at a relatively constant memory, with mostly constant average frame time for higher resolutions. Graph 3.2 implies that this algorithm may be better suited for computer graphics applications where designers intend for a constant frame rate for high resolution applications. This could apply for high resolution monitors or virtual reality devices where a constant frame rate is essential.

Using NURBS for surflet generation results in constant performance regardless of the distance of the model from the camera. Therefore using surflets can drastically improve the performance of ray tracing on devices. This study proves that it is highly likely surflets will greatly reduce frame time for global illumination in real time applications and that more research should be conducted on global illumination with freeform surfaces.

Computations for the vector based iteration is faster than the traditional planar based iteration. This implies that minimal data structures are likely to have more performance. Changing the offset based on the environment variables such as the UV position, XYZ position, and ray vector may be beneficial in reducing the number of steps of the modified Newton iteration method. This would result in the computation time for this method decreasing, improving performance as average frame time would decrease.

In conclusion, this study implements a surfel based algorithm for real time rendering that does Newton iteration for ray tracing intersections on freeform surfaces. This study has been successful in the context of research done for the rendering of freeform surfaces. However because the NURBS raytracer performs lower with freeform models with a higher vertice count, using the NURBS raytracer should be used instead of traditional raytracing if a model can be represented with lower vertices with NURBS than with tessellation. NURBS ray tracing should be used when accuracy is key, so therefore NURBS may be better suited for models close to the camera view.

5 Works Cited

[1] Hanspeter Pfister, Matthias Zwicker, Jeroen Baar, and Markus Gross. 2000. Surfels: Surface Elements as Rendering Primitives. Proceedings of the ACM SIGGRAPH Conference on Computer Graphics (May 2000).

[2] Oliver Abert, Markus Geimer, and Stefan Muller. 2006. Direct and fast ray tracing of NURBS surfaces. Symposium on Interactive Ray Tracing (September 2006), 161–168. DOI:https://doi.org/10.1109/RT.2006.280227

[3] V. Chandrasekaran, M.B. Wakin, D. Baron, and R.G. Baraniuk. 2004. Surflets: a sparse representation for multidimensional functions containing smooth discontinuities. In International Symposium onInformation Theory, 2004. ISIT 2004. Proceedings., 563-. DOI:https://doi.org/10.1109/ISIT.2004.1365602

[4] Henrik Halen, Andreas Brinck, Kyle Hayward, and Xiangshun Bei. 2021. Global Illumination Based on Surfels. In SIGGRAPH 2021. https://www.ea.com/seed/news/siggraph21-global-illumination-surfels

[5] Direct Rendering of Trimmed NURBS Surfaces. Christian Lessig. University of Toronto Dynamic Graphics Project. https://www.dgp.toronto.edu/public_user/lessig/talks/talk_alias_final.pdf